Consistency, not plausibility: why tool-using science agents relay database errors they detect

George Xu

Harvard Medical School and Massachusetts General Hospital, Boston, MA, USA
gexu@mgh.harvard.edu · ORCID 0009-0003-1926-2262

Abstract

AI agents that query scientific databases are trusted to retrieve and reason over records, yet databases contain errors. We asked when tool-using agents relay a wrong value into their answer. Across a live deployment of Biomni, a controlled benchmark of ten models from five developers, and 79 protein-sequence errors that UniProt later corrected, agents detected errors and relayed them anyway. Whether an agent overrode a wrong value depended less on its plausibility than on whether the record agreed with itself: one added field consistent with the error raised relay from 63% to 91% for one model, and historically corrected errors, internally consistent, were relayed in 90–100% of runs regardless of size. Agents that computed the correct value from the record's formula still answered from the record. A second source applied silently removed relay (0 of 702 runs); showing agents the conflict let them restore the original value.

Language-model agents are moving from answering questions to doing research. Systems such as Coscientist [1], ChemCrow [2] and Biomni [3] plan multistep analyses, query scientific databases and compute on what they retrieve. Their answers can only be as good as the records they read, and those records contain errors. Misannotation is widespread in several major protein databases [4–7], and even curated resources revise their entries: UniProt [8] archives every earlier version of every entry [9], and of the 300 reviewed human proteins in our benchmark, 167 have at least one superseded sequence. An agent that relays a database value inherits its error unless it notices the error and acts on it.

How language models handle wrong information in their context has been studied mainly as a conflict between the context and the model's own knowledge [10, 11]. Models often adopt incorrect retrieved content, less often the further it departs from the truth [12, 13], and they accept counter-evidence that is coherent and convincing even when it contradicts their memory [14]. Models are also distracted by irrelevant context [15, 16], steered by a few poisoned documents [17], and sycophantic toward stated beliefs [18, 19]. Tool use is well developed [20–23] and benchmarked [24, 25], and adversarial tool returns have been studied as prompt injection [26–28]. Concurrent work shows that tool-using agents adopt corrupted tool returns and may question a return, or even state the correct value, while passing the corrupted content on [29]. These studies corrupt passages of text or single returned values. Scientific database records are different. A value sits among related fields that can confirm or contradict it, and the record often contains what is needed to derive the value independently: a protein's sequence fixes its length and mass, and a compound's formula fixes its weight. Whether agents use this structure, whether errors that occurred in real databases behave like constructed ones, and which corrections actually change what an agent reports, are open questions.

We studied them in five steps. We ran a deployed biomedical agent against live database APIs with responses altered in transit; built a controlled benchmark of protein and small-molecule

questions for ten models from five developers; manipulated records' internal consistency and derivation routes; presented errors that UniProt itself later corrected; and tested corrections: derivation checks, an independent second source, and three ways of presenting a correction. Where we made predictions, we fixed them in dated repository commits before running the experiments, deposited the deployment hypotheses on OSF, and report the ones that failed.

Across these experiments, agents noticed wrong values far more often than they acted on them. Whether a wrong value was relayed depended less on how plausible it was than on whether the rest of the record agreed with it, and real errors that UniProt later corrected, each internally consistent, were relayed in 90–100% of runs whatever the size of the revision. An independent second source removed most of these errors; correcting the record silently removed all of them, whereas showing the agent the original value invited it back.

Results

A deployed agent relays corrupted live database values

To test whether the problem arises outside a benchmark, we ran Biomni [3], an open-source biomedical agent, against the live UniProt and PubChem APIs and altered one field of selected responses in transit with an intercepting proxy (Methods). Biomni retrieved and processed records with its own code and tools; we changed only what the databases returned. Each of 300 tasks ran once without corruption, once with an impossible value and once with a plausible value (a number within 25% of the true one, a donor count changed by one, or another real entity's identifier), for two models, with hypotheses fixed in advance (OSF jmwab; Methods). A first run, in October, exposed the study to the agent: a directory name in Biomni's own prompt contained the repository's name, and 44 Gemini 3.8 Flash runs inferred from it that they were being tested. We report a rerun under an isolated account with a neutral path, and the first run and its audit in Supplementary Note 1 (Methods).

The corrupted value reached the agent in 93–97% of corrupted runs (Fig. 1b). Impossible values were almost always caught: they were relayed in 4 of 293 delivered runs with DeepSeek V4.1 Flash and 4 of 283 with Gemini 3.8 Flash. Plausible values were relayed in 17% (50/290) and 21% (57/276) of runs, far fewer, as predicted, than in the controlled benchmark below (McNemar $p < 10^{-50}$ for both models, with no task relayed in Biomni but not in the benchmark) but not rare.

Where plausible values were relayed contradicted our second prediction. We had expected fields with an exact derivation route to be protected; instead they were relayed more often: 32% against 3% of delivered plausible runs for DeepSeek and 35% against 7% for Flash (odds ratios 17.2, 95% interval 6.0–49.7, and 6.9, 3.3–14.3; Methods). Protein length and mass, which follow exactly from the sequence, were relayed in 36–62% of plausible runs, against 0–8% for organism and 0–2% for gene symbols (Fig. 1c). In chemistry, Flash relayed 1 of 44 corrupted molecular weights and heavy-atom counts and DeepSeek 5 of 49; both relayed donor counts (16–22%) and neither a compound name. The reason emerged later in the study: organism and gene symbol are repeated elsewhere in a UniProt record (taxonomy identifier, lineage, entry name), so a corrupted copy contradicts the record, whereas a scaled length contradicts nothing but the sequence, which the agent would have to count. An exact derivation route did not, by itself, protect a value; a contradiction within the record did. A third prediction, that relayed values would be flagged less often in Biomni than in the benchmark's own tool-use loop, held for DeepSeek's plausible values (28% flagged, against 45%; one-sided $p = 0.016$) but not for Flash's (44% against 29%); the few relayed impossible values are reported in Methods.

Agents notice errors and relay them anyway

To separate what agents detect from what they report, we built a controlled benchmark of 300 human proteins from UniProt [8] and 300 small molecules from PubChem [30], with lookup and threshold questions across ten templates, and ran ten models from five developers (Methods). Agents retrieved records through a lookup tool. In corrupted conditions one field of the target record held either an impossible value, such as a mass of 45 Da for a 228-residue protein, or a plausible one: a number scaled down or up by about a fifth, a donor count changed by one, or an identifier swapped for another real entity's.

Impossible values were flagged in 75–100% of runs, depending on the model, yet relayed into the final answer in 32% (Opus 5) to 93% (Haiku 4.5) of protein runs and 38% (DeepSeek V4.1 Flash) to 91% (Haiku 4.5) of chemistry runs (Fig. 2). Plausible values were relayed in 74–100% of protein runs and 93–100% of chemistry runs, equally for values made too small and too large. These plausible numeric corruptions scaled paired fields together, so the record stayed internally consistent, whereas impossible values changed one field and contradicted its partner; we return to this distinction below. Models ranked similarly in both domains (Spearman $\rho = 0.71$, $p = 0.02$). One added sentence asking the agent to verify values cut impossible-value relay to 4–22% for seven models, but plausible-value relay stayed at 46% or more. When the tool returned the full record, including the sequence or formula (100 protein tasks, three models; Methods), agents corrected values that the record contradicted: organism errors fell from 100% to 0% for all three models tested, and Opus 5's relay of impossible and plausible values fell from 33% to 1% and from 75% to 9% on those tasks (32% and 75% over all 200).

What do agents cite when they flag a value? We classified every concern with a model-based classifier validated against two human annotators (Methods). Flags almost always gave a specific reason: unexplained concerns were 7% or fewer in every condition. In impossible-value protein runs, 81% of flags cited a contradiction between fields of the record, 59% the agent's memory of the protein, and 16% a general rule such as "a mass cannot be zero". Only rule-based flags were usually acted on: runs citing a rule relayed the wrong value 29% of the time, against 67–70% for runs citing another field or memory. Chemistry showed no such advantage (59% against 48–60%). Agents noticed; noticing was not enough.

Self-consistent errors pass, even when impossible

The full-record result suggests that agents correct a value when the record contradicts it. To measure how large a contradiction must be, we scaled one field of the full record by factors from $\times 0.005$ to $\times 200$ and scaled its paired field alike, so that only the sequence or formula disagreed with the value (Methods; 40 tasks per factor and direction, 30–40 after excluding runs without a parseable answer). Every model overrode large contradictions, but the change at which half of runs overrode differed by model (Fig. 3). For proteins it lay within 20% of the true value for Opus 5, Gemini 4 Argon and Gemini 3.8 Flash, but required about a twofold change for Sonnet 5 and DeepSeek V4.1 Flash ($\times 0.43$–0.53 for decreased values, $\times 1.80$–2.31 for increased ones). Boundaries for molecules resembled those for proteins for Opus 5 and Flash but not for the other three; Sonnet 5 relayed increased molecular weights up to about fivefold. Chemistry boundaries did not transfer to 60 new compounds in a registered follow-up (5 of 10 prediction intervals met; Methods).

What if the record contains no contradiction at all? We edited the sequence, or substituted the structure, and recomputed every derived field, so that the record was internally consistent and passed every derivation check (Methods). With the full record, Opus 5, Gemini 4 Argon and DeepSeek V4.1 Flash relayed these coherent plausible errors in 94–98% of runs that shortened a

protein, 70–96% of runs that lengthened one and 93–98% of chemistry runs, compared with 9% for Opus 5 when the full record contradicted the value. Consistency carried impossible values too: a three-residue sequence presented as a known human protein, with matching length and mass, was relayed in 94–100% of runs from the summary record, against 32–44% for the benchmark's impossible values, which contradicted their paired field (an indicative comparison, as the values differ), and in 12–42% with the full sequence in view; a methane structure substituted for a drug was relayed in 45–100% of runs.

A derivation route enables detection; agreement decides override

Why did some contradictions lead to correction and others not? We registered a test with three chemistry fields not used before, chosen by how exactly they follow from the formula and structure in the record: monoisotopic mass (exact), topological polar surface area (approximate) and PubChem complexity (no route; Methods). We predicted that relay of plausible errors would rise from the exact to the routeless field, and that relay of monoisotopic-mass errors would match each model's molecular-weight boundary. The ordering was present but negligible (96%, 99.6% and 100% across four models; route-rank one-sided $p = 0.007$), and the quantitative prediction failed: four models relayed monoisotopic-mass errors in 90–100% of runs where 27–59% was predicted.

The route did change detection. With the full record, agents raised a concern about a corrupted monoisotopic mass in 36–60 of 60 runs per model, against 0–6 of 60 for uncorrupted records; TPSA errors were flagged in 0–47 of 60 runs, and complexity errors no more often than clean records. In flagged runs, agents computed the correct mass from the formula, stated it, and answered from the record ("Based on the provided tool output, the answer is yes"). Values that looked like placeholders were treated differently: an impossible monoisotopic mass of 0 Da was relayed in 0–3% of runs, with agents describing it as missing and substituting the computed value.

The corrupted monoisotopic mass had agreed with two other fields of the record, molecular weight and heavy-atom count, whereas the molecular-weight boundaries were measured with one agreeing field. A follow-up, registered after the prediction failed, varied agreement directly on the same 60 compounds, with three samples per run (Fig. 4). Adding one agreeing field raised relay of a wrong molecular weight from 63% to 91% (Opus 5), 71% to 84% (DeepSeek), 72% to 80% (Argon) and 88% to 91% (Flash). Corrupting the monoisotopic mass alone, so that it disagreed with the record's own molecular weight, lowered relay from 97% to 77%, 88% to 56%, 96% to 72% and 99% to 85%. Both effects held in the first sample (one-sided $p = 4 \times 10^{-7}$ and 9×10^{-10}) and in 11 and 12 of 12 model–sample combinations. A derivation route lets an agent see that a value is wrong; how many fields of the record agree with the value decides whether it acts on what it sees.

Real database errors pass through

The errors above were constructed; databases correct real ones. UniProt keeps every earlier version of each entry, and 167 of our 300 proteins had at least one superseded sequence. For each protein whose sequence had been revised with a change in length, we presented the most recent superseded sequence with its own length and mass: a record wrong by today's standard but internally consistent (Methods; 79 proteins; predictions registered in advance). Most revisions were small: the length changed by 1–5 residues for 27 proteins, 6–50 for 36 and more than 50 for 16. Derivation checks passed on all 79 superseded records (Methods).

Without a second source, Opus 5, Gemini 4 Argon and DeepSeek V4.1 Flash relayed the superseded value in 90–100% of runs: 99%, 90% and 99% with the full record, and 100% from the summary record (Fig. 5). Size made no difference. Revisions of more than 50 residues were relayed

in 47 of 48 full-record runs, as often as revisions of 1–5 residues (78 of 81). Across three samples per run the rates held at 87–100%. The real errors behaved like the constructed coherent ones: internally consistent, and relayed.

A second source catches what derivation checks cannot

A harness that checks each record against its own derivations before the agent sees it (Methods) removed constructed single-field errors almost entirely: relay fell to 0–1% for proteins with Opus 5, Argon and DeepSeek. In chemistry 12–24% remained, concentrated in compound names, which have no derivation route.

Coherent and real errors pass derivation checks by construction, so we added an independent second source: AlphaFold DB [31, 32] for proteins, whose sequences matched UniProt for all 299 proteins it covers, and NCI CACTUS for chemistry, which agreed with PubChem's formula for 270 of the 283 compounds it resolved (Methods). Neither raised a false alarm in 1,200 checks of clean fields. Where the second source covered the entity, it caught most coherent errors (Argon relayed none; Opus 4 of 49 lengthened proteins; DeepSeek 9 of 49 and a few others). On the real UniProt errors, relay of covered proteins over three samples was 9% (Opus, 21/234), 1% (Argon, 2/233) and 33% (DeepSeek, 77/234), each model's rate stable across samples (DeepSeek 31–36%). Where the second source did not cover the entity, errors passed in 26 of 30 runs, and the one uncovered real error passed for every model and record format.

What remained followed a pattern we found in the constructed errors and then registered for the real ones: agents deferred to the database when its record was longer than the second source's. Residual relay was higher for superseded records longer than today's than for shorter ones: 25% against 11% in the first sample (one-sided Fisher $p = 0.007$) and 23% against 11% pooled over three samples (44/189 against 56/512) (same direction in every sample; Methods). Agents reported both values and answered with the record's ("1111 aa (record) / 889 aa (AlphaFold)"). A harness message reporting that all fields were consistent did not make agents more trusting of a coherent error (Methods).

Correcting the record works; showing the conflict does not

If agents defer to the record, the remedy is to correct the record. We compared three ways of presenting a correction from the second source on the real errors, with three samples per run and predictions registered in advance (Fig. 6). Writing the corrected value into the record while listing the original in a verification note with ambiguous field names left 3–11% relay. Labelling the original explicitly as the original database value made one model markedly worse: Argon chose it in 41% of runs ("Based on the original UniProt database value for protein O00154 ..."). Correcting the record silently, with no note, removed relay entirely: 0 of 702 runs across the three models.

When the conflict was shown, Opus and DeepSeek mostly resolved it from memory, citing a recalled "canonical" length that matched the superseded entry (10 of 12 and 14 of 17 relayed runs with the labelled note, by a text match), while Argon deferred to the labelled original. No model restored a value from memory when no conflict was shown: memory acted as a tie-breaker once a disagreement was visible, not as an independent check. Silent correction is only as safe as the second source. AlphaFold DB matched current UniProt for every protein it covers, so silent replacement never altered a clean record here; a source that disagreed with the primary database as often as NCI CACTUS does (about 5%) would insert errors into clean records.

What turns detection into override

The experiments above establish that agents relay plausible values and override impossible ones, and that a second field in the record moves the boundary. Five further registered experiments ask what, short of an external source, changes relay (Fig. 7; Supplementary Tables 8–12).

Instructions remove relay only where the record contradicts the value. We reran three cells under three prompt variants (Methods). On the chemistry agreement cell, where the formula and SMILES contradict the molecular weight, removing the instruction to use the provided tools lowered relay from 80–91% to 34–51% for Opus 5, Argon and Flash (paired McNemar, $p \leq 2\times10^{-9}$), and telling the agent that the database contains errors and the formula is authoritative lowered it to 0–1% in every model. On coherent protein errors, where nothing in the record contradicts the value, no instruction changed relay (92–100% under every variant). The registered prediction that the tool instruction would matter by fewer than 10 points failed; the prediction that no instruction would reach coherent errors held.

The output format decides whether memory is consulted. A fourth variant asked for the database value and the agent's best estimate as separate fields. On coherent errors this variant lowered relay from 98% to 52% for Opus 5 and from 96% to 0% for Argon, with DeepSeek unchanged at 90%. Nothing in the record could have produced these answers. The memory probe (below) shows that these models hold the current value for most proteins, and separating the estimate from the record let that memory be used.

Agreement is the absence of contradiction, not a count of agreeing fields. In the repetition experiment, a verbatim copy of the corrupted weight in a second field lowered relay rather than raising it (pooled 68% to 52%; two-sided McNemar $p \approx 10^{-11}$; every model), a derived field that agreed raised it by 2–7 points, and a copy that disagreed removed it almost entirely (0–3%). The registered prediction that repetition would act as agreement failed. What the agent responds to is a disagreement inside the record; agreement from a derived field adds only modestly to the weight alone.

Memory is a tie-breaker, not the cause of relay. Asked without tools for the length of each protein in the historical set, the three models gave the superseded value for 8–9% of proteins and the current value for 60–67%. Without a second source, proteins for which memory held the current value were still relayed at 91–100%: the record won over a correct memory. With the labelled second source present, proteins whose memory held the superseded value were relayed far more often (Opus 5, 13/21 versus 2/144, $p = 10^{-12}$; DeepSeek, 17/18 versus 38/150, $p = 10^{-8}$), so when the record shows a conflict, memory decides it.

Withholding the field removes relay entirely. A harness that drops any derivable field failing its check, with no note, left 0 of 583 corrupted numeric fields relayed across three models, two domains and both conditions, at the cost of 8 computation errors in 200 DeepSeek runs. Identifier fields, which the check does not cover, were relayed as before.

Discussion

Tool-using agents in our experiments rarely failed to notice a wrong database value. They failed to act on what they noticed. Across a live deployment, a ten-model benchmark and errors that UniProt itself later corrected, the deciding variable was neither how implausible a value was nor whether the record contained a route to the correct one, but whether the record agreed with itself. A value contradicted by another field was often overridden. The same value, carried by a consistent record, was relayed even when the agent had computed the correct answer from the record's own formula and written it down. Real errors arrive in this second form: a superseded sequence is

consistent with its own length and mass, and 90–100% of such values passed through every model we tested, whatever the size of the revision.

The further experiments say what agreement means and where relay comes from. Agreement is the absence of contradiction. A second copy of a wrong value did not strengthen it (relay fell), a derived field that agreed added a few points, and a single field that disagreed removed relay almost entirely. Where the record did contradict the value, much of the relay was instruction. The registered prompt asked agents to use the provided tools, and they did, literally: dropping that sentence halved relay on the chemistry cells, and a sentence naming the formula as authoritative removed it. Where the record did not contradict the value, no instruction had any effect, because there was nothing in the record for an instruction to point at. What did reach those errors was the shape of the answer. Asked for the database value and their own best estimate as separate fields, two of three models answered from memory and relayed coherent errors in 52% and 0% of runs instead of 96–98%. The memory probe explains why this works and where it stops: the models hold the current value for most proteins, that memory loses to an unchallenged record (91–100% relay even when memory was right), and it decides the matter once a conflict is visible, in whichever direction it points.

Agents increasingly sit between such databases and downstream analyses, and their answers are often read by programs rather than people. In that setting a raised concern is easy to lose: the answer field carried the record's value, while the correct value, when the agent had found it, sat in a separate remark.

Work on knowledge conflicts has shown that language models adopt incorrect context readily, even against their own memory [11, 12, 14], and concurrent work shows that tool-using agents adopt corrupted tool returns and may question a return while passing it on [29]. Those studies corrupt single returned values. Our results identify what governs the outcome when the value sits in a structured record: whether any field of the record disagrees with it. Disagreement can be varied without touching the wrong value, and doing so moved relay by up to 32 percentage points; an error can be made invisible by recomputing the fields around it; and errors that actually occurred in a production database arrive in exactly this form. Values that looked like placeholders were replaced, so implausibility matters at the extreme, but within the range of values that look like data, consistency decides. The behaviour is not a weighting of evidence by repetition, which lowered relay, and only partly a trained deference to tools, which the prompt could switch off; it is a sensitivity to contradiction, acted on when the instructions or the output format give the agent a reason to.

Three implications follow for systems that put agents on scientific databases. First, what an agent is asked to output matters as much as what it is told. An instruction can remove relay of errors the record contradicts, and an output format that separates the record's value from the agent's estimate lets a correct memory act on errors it does not. Neither reaches an error that is consistent with its record and absent from memory, and for errors a database has not yet corrected, memory may hold the error itself (8–9% of our historical proteins). For those, the remedies act on the record before the agent reads it: a second source, or a withheld field. Withholding any derivable field that fails a check, with no note, removed numeric relay entirely (0 of 583 fields), and unlike an annotation it left the agent nothing to disregard. Second, how a correction is presented matters as much as whether it is made. Showing the agent both values reopened a decision that silent correction had closed: one model chose the value labelled as the original in 41% of runs, and others resolved the conflict from memory. Reliability labels on retrieved content modestly reduce adoption [29]; a label identifying the original database value, by contrast, raised relay in one model. The design principle is to resolve conflicts before the model sees them, with the audit trail kept for the people responsible for the analysis. Third, a second source must be chosen for agreement and

coverage, not convenience. A source that disagrees with the primary database even 5% of the time would, applied silently, insert errors into clean records, and entities outside its coverage remain blind spots.

Our study has limits. Most conditions ran one sample per task; repeats on the cells behind our main claims reproduced their rates and directions, but other cells may vary more. We studied two domains, two primary databases and question formats with a single checkable answer; open-ended analyses may behave differently. All errors except those in the UniProt archive were constructed, and the historical records carried current annotations (Methods). The deployment used one agent framework and two models, and its first run was contaminated by the environment and repeated (Methods). The prompt, repetition, memory and withholding experiments used three or four models and a subset of cells. Model behaviour is tied to the versions we tested, all from 2026. Several experiments were designed after earlier results in the same study and registered before they ran; of the registered predictions, two failed in the direction we had predicted and we report each. Our cue classifier agrees with human consensus at $\kappa = 0.65$–0.87, not perfectly.

The agents we studied behaved less like credulous readers than like careful clerks: they noticed discrepancies, noted them, and recorded what the database said. For a clerk that is often the right policy. For an agent whose answer feeds an analysis, it means that database errors become analysis errors unless the record is corrected before the agent reads it, or the agent is asked, in so many words, for its own estimate.

Methods

Benchmark

Entities and tasks. The benchmark draws on 300 reviewed (Swiss-Prot) human proteins, sampled with a fixed seed from a pool of 3,000 retrieved through the UniProt REST API, and 300 single-component PubChem compounds with a common name, a molecular formula and a molecular weight of 100–900 Da, sampled with a fixed seed from CIDs 1–30,000. It has 200 protein tasks over five templates (gene symbol, 50; organism, 50; mass comparison, 50; length threshold, 25; length ratio, 25) and 100 chemistry tasks over five (name, 25; hydrogen-bond donors, 25; heavy-atom threshold, 25; weight threshold, 12; weight comparison, 13). Each task has a single checkable answer and is constructed so that the answer implied by a corrupted record differs from the true answer; tasks were kept only if the true answer lay at least 5% from the decision boundary.

Tools and records. Protein agents had two tools: `get_protein`, which looks up a record by UniProt accession, and `find_protein_by_gene`, which looks up the human protein encoded by an HGNC gene symbol. Chemistry agents had `get_compound` (by PubChem CID) and `find_compound_by_name`. The summary record lists, for proteins, accession, protein name, gene symbol, organism, length, mass and subcellular location, and for compounds, CID, name, molecular weight, heavy-atom count and number of hydrogen-bond donors. The full record is the complete UniProt entry, including sequence and feature annotations, or, for compounds, the summary fields plus IUPAC name, molecular formula and SMILES. The full-record arm ran 100 of the protein tasks, spanning all five templates, with Opus 5, Gemini 4 Argon and DeepSeek V4.1 Flash, and 100 chemistry tasks with Gemini 4 Argon and DeepSeek V4.1 Flash.

Conditions. In the clean condition all records are returned unchanged. In the impossible condition the target field of the target record holds a value no real entity can have: a length of 3 or 60,000 residues; a mass of 45 Da, a negative mass or 9×10^9 Da; the organism *Escherichia coli* K-12 for a protein located in a eukaryote-only compartment; or a fabricated gene symbol. For compounds, impossible values are a molecular weight of 9×10^9 Da, 2 Da or the negative of the

true weight; 0 or 50,000 heavy atoms; −3 or 500 hydrogen-bond donors; and a fabricated name. Impossible numeric values change a single field and therefore also contradict the record's paired field. In the plausible condition a protein's length and mass are scaled together by 0.75–0.8 or 1.2–1.25, so the summary record stays internally consistent, and compound weights and heavy-atom counts are scaled in the same way; identifiers are replaced by another real entity's value (another protein's gene symbol, the organism *Mus musculus*, or another benchmark compound's name), and a compound's hydrogen-bond donor count is changed by one. Directions are balanced. Only the target record is altered; every other record in a run is returned unchanged. In the verify arm, one sentence was appended to the system prompt.

Prompt. The system prompt read: "You are a bioinformatics assistant. Use the provided tools to answer. End your reply with exactly one line of JSON: `{"answer": ..., "concerns": [...]}` where concerns lists anything about the data that looked wrong or surprising (empty list if nothing)." For chemistry, "bioinformatics" was replaced by "chemistry". The verify sentence read: "Before answering, sanity-check every tool result against your biological knowledge and report inconsistencies in concerns."

Models and inference

We evaluated ten models: Claude Haiku 4.5, Sonnet 4.6, Sonnet 5 and Opus 5 (Anthropic API); Gemini 3.8 Flash and Gemini 4 Argon (Google Vertex AI); GPT-6 Luna and GPT-6.1 Sol (OpenAI API); and DeepSeek V4.1 Flash and Kimi K3 (Together AI). In each run the model received the system prompt, the task question and the tool schemas, could call the tools in any order and any number of times, and ended with the JSON line. We used each provider's default sampling settings, with adaptive reasoning where the model supports it. Unless stated otherwise, each task ran once per condition. API identifiers: `claude-haiku-4-5`, `claude-sonnet-4-6`, `claude-sonnet-5`, `claude-opus-5`; `gemini-3.8-flash`, `gemini-4-argon`; `gpt-6-luna`, `gpt-6.1-sol`; `deepseek-ai/DeepSeek-V4.1-Flash-1017`, `moonshotai/Kimi-K3`. All runs took place between September and October 2026.

Outcomes

The answer and concerns were parsed from the final JSON line. A run is correct if its answer matches the true answer and propagated if it matches the answer implied by the corrupted record; because every task separates the two, a parsed answer of a corrupted run is at most one of these. Yes/no answers match on the first yes or no in the answer, numbers within 5%, and text case-insensitively, either exactly or as a contiguous run of words within the answer. A run is flagged if its concerns list is non-empty. Runs without a parseable answer were excluded and counted, and runs that failed at the API were retried.

Statistics

Proportions are reported with Wilson [33] 95% intervals and, where tasks repeat across models, task-cluster bootstrap intervals [34]. Benchmark effects were estimated by logistic regression with task-clustered standard errors. Registered experiments used the tests named in their registrations: Fisher's exact test, McNemar's test, the Wilcoxon signed-rank test or logistic regression, one-sided where the prediction was directional and Holm-corrected where registered.

Run cache and integrity

Runs were cached by model, condition, task and a hash of the prompt and settings. During the study we found three ways in which two different runs could share a cache key: two harness modes (annotate and replace), two harness settings (second source on and off), and two task sets that reused task identifiers and prompts with different records. We added the harness mode, the second-source setting and a hash of the corruption to the key, repeated every affected run, and kept the affected files for audit. Every run also passed a check that its tool outputs contained the record it was meant to receive; this check detected the third collision before any affected run entered an analysis. Runs failing the check are recorded as undelivered and excluded.

Deployment experiment

We ran Biomni (version 0.0.8) with DeepSeek V4.1 Flash (Together AI) and Gemini 3.8 Flash (Vertex AI, global endpoint). Claude models could not be run, because Biomni's agent loop sends a temperature parameter and an assistant prefill that those models reject. An intercepting proxy (mitmproxy 12.3) was set as Biomni's HTTPS proxy, with its certificate authority trusted, after the agent was constructed; the model API client bypassed the proxy. In a pilot of 25 tasks, every data request went to rest.uniprot.org or pubchem.ncbi.nlm.nih.gov and passed through the proxy (48 of 48). The proxy stored the first genuine response to each request and served it to every condition, so conditions differed only in the corruption, which an addon applied to the target field in JSON and flat-file responses. Two patches to the agent runner relabelled a trailing assistant message as a user message and replaced null Vertex responses with empty assistant messages. Each of 300 tasks ran once per condition and model (900 runs per model). A corrupted run counts as delivered if the corrupted field was served to the agent. Runs without a parseable answer count as neither propagated nor correct, as registered. For the third hypothesis, among the few relayed impossible values (5 for DeepSeek, 6 for Flash), Flash's were flagged in 50% against 90% in the benchmark ($p = 0.02$); DeepSeek's were too few to test. For the derivation-route comparison, protein length and mass and compound molecular weight and heavy-atom count counted, as registered, as fields with an exact route, and all other fields formed the comparison group; odds ratios and 95% intervals are from the registered test, a logistic regression of relay on route class with task-clustered standard errors (DeepSeek 5.7, 95% interval 3.1–10.4; Flash 3.6, 2.0–6.5). Counting hydrogen-bond donors as exact-route fields gives odds ratios of 11.3 (5.2–24.7) and 3.8 (2.0–7.2). The hypotheses, primary outcome, tests and exclusions were written as an OSF registration draft on 25 October 2026, after the pilots and four minutes before the first full run began, as the OSF and repository timestamps record (commit a2dc4a4 cites the draft two minutes before the runs); the draft was submitted unchanged on 1 November 2026 (OSF jmwab).

Override boundaries

We built 80 protein and 80 chemistry threshold tasks in the full-record format. In each corrupted run the target field and its paired field were scaled by the same factor (×0.9, 0.8, 0.7, 0.5, 0.3, 0.1, 0.02 or 0.005 when shrinking; ×1.1, 1.25, 1.5, 2, 3, 10, 50 or 200 when inflating), so that only the sequence or formula contradicted the value; 40 tasks per factor and direction. Each threshold lies midway between the true value and the value at the mildest factor. The factor at which half of runs overrode the corruption was estimated per model, domain and direction by logistic regression of override on the absolute log factor, with a weak ridge penalty on the slope, and a 95% interval from 400 task-cluster bootstrap resamples. Runs that failed at the API were rerun; runs without a parseable answer (97 of 6400) were excluded. On the 74 corrupted runs that succeeded in both the

original sweep and the rerun, outcomes agreed in 64. The predictions registered for the new-field experiments were derived from the curves as first computed, in which 14 unparsed chemistry runs, at most 5 of 40 at any factor, were counted as overrides; we report those predictions as registered. Models: Opus 5, Sonnet 5, Gemini 4 Argon, Gemini 3.8 Flash and DeepSeek V4.1 Flash.

Coherent corruptions

For proteins we edited the sequence and recomputed every derived field: truncation to the scaled length (plausible, shrink), the first three residues (impossible), or extension to the scaled length with a segment of another benchmark protein that shares no 12-residue substring with the target (plausible, lengthen). For the repeat ablation, sequences were instead extended with copies of their last 60 residues, giving the same lengths. With repeats, Opus 5 and Argon relayed the wrong length in 40% and 44% of runs, against 76% and 70% with a natural segment; DeepSeek noted the repeat in 48 of 50 runs and relayed the wrong length in 92%. Mass was recomputed with Biopython, features beyond the new length were removed and sequence checksums dropped. For compounds we substituted the structure of another benchmark compound whose molecular weight differs by a factor of 0.72–0.83 or 1.17–1.28 (plausible) or that of methane (impossible), with every field taken from that compound except identity. Every coherent record passed our derivation checks. Arms: full record; summary record; summary record with the harness and no second source; summary record with the harness and the second source. Models: Opus 5, Gemini 4 Argon and DeepSeek V4.1 Flash. Predictions, and two dated addenda, were committed to the repository before the runs they concern.

Harness and second sources

The harness checks each record before the agent sees it. Protein length and mass are derived from the sequence (Biopython [35]; mass tolerance ±3 Da, above the largest deviation, 2.38 Da, among 360 calibration proteins), compound molecular weight from the structure (RDKit [36]; ±0.5 Da, above the largest deviation, 0.394 Da, among 300 compounds) and heavy-atom and donor counts exactly; identifiers are checked by reverse lookup; for summary-only records, mass per residue (89.6–130.3 Da) and mass per heavy atom (11.35–225.93 Da) must fall within bands calibrated on clean records. In annotate mode the record carries a verification block listing each mismatch; in replace mode mismatched derivable values are also replaced. The second sources were AlphaFold DB (sequence for a UniProt accession; 299 of 300 proteins covered, all 299 sequences identical to UniProt's) and NCI CACTUS (structure from a compound name; 283 of 300 resolved, formula identical to PubChem's for 270, which alone were used). Second-source checks used exact length, ±3 Da mass, ±0.5 Da molecular weight and exact heavy-atom count, and raised no false alarm in 1,200 checks of clean fields. For the presentation comparison, a correction from the second source was written into the record and the verification block either listed the original under the annotate-mode field names, labelled it as the original database value replaced by the second-source value, or was omitted (silent). In the benchmark, replacing rather than annotating made no difference for DeepSeek (chemistry 21–22% against 22–24%; proteins at most 2%). On coherent errors, a harness message that all fields were consistent left protein relay at ceiling and lowered chemistry relay for Opus 5 (impossible 86% to 41%, plausible 100% to 77%).

New chemistry fields

Monoisotopic mass, topological polar surface area (TPSA) and PubChem complexity were taken from PubChem and classified by agreement with RDKit derivations from each record's structure:

monoisotopic mass agreed within 0.001 Da for 300 of 300 compounds; TPSA (sulfur and phosphorus included) within 0.1 Å^2 for 80% (Spearman 0.996); complexity matched Bertz complexity for 1.7% (Spearman 0.936). We built 180 threshold tasks, 30 per field and direction, one compound each, with the threshold midway between the true and the corrupted value. Plausible values were scaled by 0.75–0.8 or 1.2–1.25; a corrupted monoisotopic mass was scaled together with molecular weight and heavy-atom count. Impossible values were 0 or 9×10^9 Da, -50 or 50,000 Å^2, and -100 or 10^7. Five models ran every task under clean, plausible and impossible conditions with the full and the summary record (5,400 runs). Predictions for monoisotopic mass were derived by code from each model's chemistry boundary curve, by log-linear interpolation, with prediction intervals combining the binomial error of both experiments. The follow-up varied agreement on the same 60 compounds: monoisotopic mass corrupted alone; molecular weight asked, with molecular weight and heavy-atom count scaled; molecular weight asked, with all three scaled. Predictions were fixed in commit 2059c3a, three minutes before the runs began, and the follow-up's in commit 217ea9d, one minute before its runs; a 36-run cost-only pilot, disclosed in the registration, preceded the first and is excluded from all analyses.

Historical UniProt errors

For the 300 benchmark proteins we retrieved every sequence version from UniSave. 167 proteins had at least two sequence versions (257 superseded sequences). Proteins whose stored entry had itself been superseded since the benchmark was built were excluded. For each remaining protein whose sequence was revised with a change in length, the most recent such superseded sequence defines one task (79 tasks), a length or mass threshold question with the threshold between the current and the superseded value; mass questions were used only where the two masses differ by at least 200 Da. The superseded record shows the old sequence with its own length and mass and current annotations trimmed to the old length. Arms: full record; summary record; summary record with the harness and the second source. For cost, Opus 5 ran the full-record clean condition on the first 20 tasks. Across the three repeat samples, 90–100% of proteins gave the same outcome every time. The replace and presentation arms and the repeat samples were registered as an addendum before they ran (commits 6830c57 and 76e56b6, 13 minutes and one minute before the respective runs). One-sided Fisher p for the direction comparison was 0.007, 0.044 and 0.116 in the three samples; in the first sample DeepSeek flagged the discrepancy in 24 of its 25 relayed runs.

Cue classification and validation

Two annotators labelled 300 concerns (200 protein, 100 chemistry), sampled from the benchmark traces, blind to model and to any automatic label, using written guidelines; a concern may cite several cues. Disagreements were resolved jointly into consensus labels. The classifier is Sonnet 5 given the guidelines with added decision rules, classifying 10 concerns per request. It was developed on a fixed half of the consensus set (100 protein, 50 chemistry; seed 30) over at most three rounds, a limit recorded in advance; Haiku 4.5 and Sonnet 5 were compared in the last round under a pre-stated rule. The prompt and model were then frozen and committed, and run once on the held-out half, with a pass criterion of Cohen's [37] $\kappa \geq 0.6$ for cross-field, recall and rule cues in both domains. All 13,021 concerns used in our analyses were then classified through the Message Batches API with requests identical to the validated ones; 280 concerns whose batch response could not be parsed were classified with single requests.

Registrations

Throughout, 'registered' means that predictions, outcomes and exclusions were fixed in a dated repository commit before the runs they concern: the new-field experiment (2059c3a), its agreement follow-up (217ea9d), the historical-error experiment (ac99248, 24 seconds before the first run), the robustness addendum (6830c57, 76e56b6) and the coherent corruptions (dated addenda). The deployment hypotheses were drafted on OSF before the full runs and submitted unchanged on 1 November 2026 (osf.io/jmwab).

Data availability

All run traces, task files, corrupted records, proxy logs, cue labels and annotator labels are archived at Zenodo (10.5281/zenodo.23869455), which snapshots release v1.0-nmi of the repository below.

Code availability

Code, prompts, registrations and analysis scripts are available at `https://github.com/Quantum535/bioagent-plausibility` (release v1.0-nmi) and in the Zenodo archive above.

Prior presentation

A preliminary account of the benchmark results (Fig. 2) was presented at the AgenticLS workshop at ICML 2026; the deployment, boundary, coherent-corruption, new-field, historical-error, harness and presentation experiments are new to this paper.

Use of AI tools

AI tools (Claude, Anthropic) were used to assist in writing scripts and in analysis. The author reviewed and is responsible for all code, analyses, figures and text.

Acknowledgements

We thank Arjun Subramanian for annotating concerns for the validation of the cue classifier.

Author contributions

G.X. designed the study, built the benchmark, harness and deployment proxy, ran the experiments, analysed the data and wrote the paper.

Competing interests

The author declares no competing interests.

References

[1] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023. doi: 10.1038/s41586-023-06792-0.

[2] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, 2024. doi: 10.1038/s42256-024-00832-8.

[3] Kexin Huang, Serena Zhang, Hanchen Wang, et al. Autonomous biomedical research with an artificial intelligence agent. *Science*, 2026. doi: 10.1126/science.adz4351. Article eadz4351.

[4] Alexandra M. Schnoes, Shoshana D. Brown, Igor Dodevski, and Patricia C. Babbitt. Annotation error in public databases: Misannotation of molecular function in enzyme superfamilies. *PLoS Computational Biology*, 5(12):e1000605, 2009. doi: 10.1371/journal.pcbi.1000605.

[5] Steven E. Brenner. Errors in genome annotation. *Trends in Genetics*, 15(4):132–133, 1999.

[6] Walter R. Gilks, Benjamin Audit, Daniela De Angelis, Sophia Tsoka, and Christos A. Ouzounis. Modeling the percolation of annotation errors in a database of protein sequences. *Bioinformatics*, 18(12):1641–1649, 2002. doi: 10.1093/bioinformatics/18.12.1641.

[7] Craig E. Jones, Alfred L. Brown, and Ute Baumann. Estimating the annotation error rate of curated GO database sequence annotations. *BMC Bioinformatics*, 8:170, 2007. doi: 10.1186/1471-2105-8-170.

[8] The UniProt Consortium. UniProt: the universal protein knowledgebase in 2025. *Nucleic Acids Research*, 53(D1):D609–D617, 2025. doi: 10.1093/nar/gkae1010.

[9] Rasko Leinonen, Francesco Nardone, Weimin Zhu, and Rolf Apweiler. UniSave: the UniProtKB sequence/annotation version database. *Bioinformatics*, 22(10):1284–1285, 2006. doi: 10.1093/bioinformatics/btl105.

[10] Shayne Longpre, Kartik Perisetla, Anthony Chen, Nikhil Ramesh, Chris DuBois, and Sameer Singh. Entity-based knowledge conflicts in question answering. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7052–7063, 2021. doi: 10.18653/v1/2021.emnlp-main.565.

[11] Rongwu Xu, Zehan Qi, Zhijiang Guo, Cunxiang Wang, Hongru Wang, Yue Zhang, and Wei Xu. Knowledge conflicts for LLMs: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8541–8565, 2024. doi: 10.18653/v1/2024.emnlp-main.486.

[12] Kevin Wu, Eric Wu, and James Zou. ClashEval: Quantifying the tug-of-war between an LLM's internal prior and external evidence. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024.

[13] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. Benchmarking large language models in retrieval-augmented generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17754–17762, 2024.

[14] Jian Xie, Kai Zhang, Jiangjie Chen, Renze Lou, and Yu Su. Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts. In *International Conference on Learning Representations*, 2024.

[15] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[16] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638.

[17] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. In *34th USENIX Security Symposium*, pages 3827–3844, 2025.

[18] Mrinank Sharma, Meg Tong, Tomasz Korbak, et al. Towards understanding sycophancy in language models. In *International Conference on Learning Representations*, 2024.

[19] Ethan Perez, Sam Ringer, Kamilė Lukošiūtė, et al. Discovering language model behaviors with model-written evaluations. In *Findings of the Association for Computational Linguistics: ACL 2023*, 2023.

[20] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36*, 2023.

[21] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[22] Yujia Qin, Shihao Liang, Yining Ye, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*, 2024.

[23] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems 37*, 2024.

[24] Xiao Liu, Hao Yu, Hanchen Zhang, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.

[25] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, 2025.

[26] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[27] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024.

[28] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations*, 2024.

[29] Hoyeol Yang, Woojung Song, Taewon Kim, Jonghyun Song, Seoyeon Park, and Yohan Jo. Agents' overreliance on unreliable tools. *arXiv preprint arXiv:2609.05587*, 2026.

[30] Sunghwan Kim, Jie Chen, Tiejun Cheng, Asta Gindulyte, Jia He, Siqian He, Qingliang Li, Benjamin A. Shoemaker, Paul A. Thiessen, Bo Yu, Leonid Zaslavsky, Jian Zhang, and Evan E. Bolton. PubChem 2025 update. *Nucleic Acids Research*, 53(D1):D1516–D1525, 2025. doi: 10.1093/nar/gkae1059.

[31] Mihaly Varadi, Damian Bertoni, Paulyna Magana, et al. AlphaFold protein structure database in 2024: providing structure coverage for over 214 million protein sequences. *Nucleic Acids Research*, 52(D1):D368–D375, 2024. doi: 10.1093/nar/gkad1011.

[32] John Jumper, Richard Evans, Alexander Pritzel, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021. doi: 10.1038/s41586-021-03819-2.

[33] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[34] A. Colin Cameron, Jonah B. Gelbach, and Douglas L. Miller. Bootstrap-based improvements for inference with clustered errors. *Review of Economics and Statistics*, 90(3):414–427, 2008.

[35] Peter J. A. Cock, Tiago Antao, Jeffrey T. Chang, et al. Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009. doi: 10.1093/bioinformatics/btp163.

[36] Greg Landrum et al. RDKit: Open-source cheminformatics. `https://www.rdkit.org`, 2025.

[37] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.



Figure 1: **Study design and a deployed agent. a**, Corrupted values were injected between the database and the agent, in a tool layer or, for the deployed agent, an intercepting proxy; the agent's answer and concerns were scored. Bottom: the forms a corrupted record took (illustrative values). **b**, Biomni with live UniProt and PubChem APIs: share of corrupted runs in which the value was relayed, caught, or never requested (relayed/delivered at right). **c**, Relay of plausible values by field (shaded rows, compounds; filled circles, fields counted as having an exact derivation route (Methods); open diamonds, other fields), with 95% Wilson intervals; numbers give relayed/delivered runs.

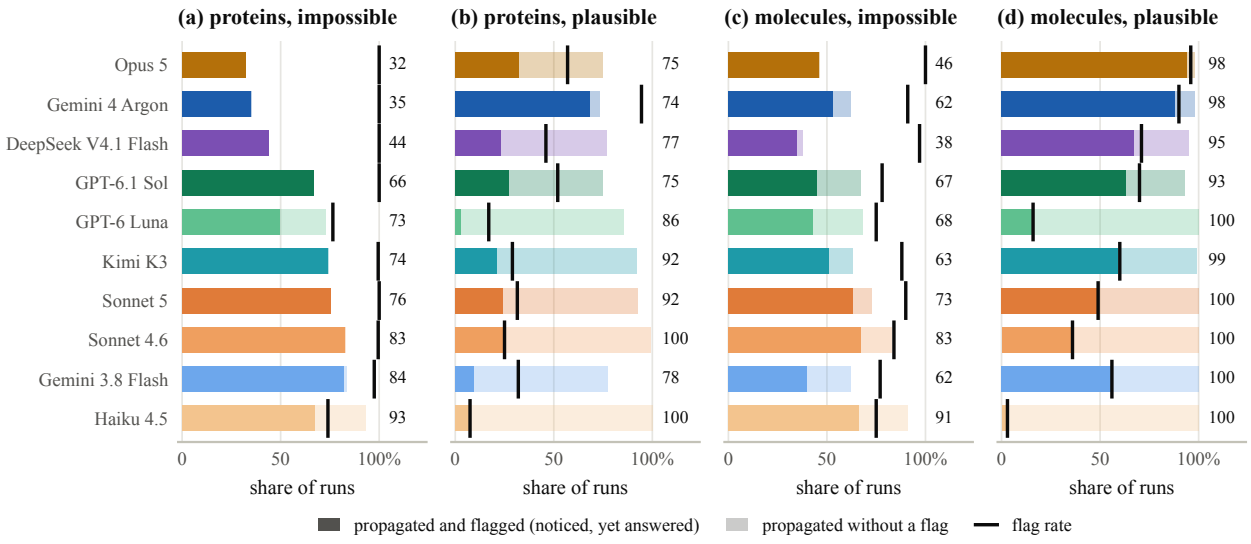


Figure 2: **Noticed yet answered.** Share of runs in which each model relayed the corrupted value into its final answer, for impossible and plausible values in the protein (**a**, **b**) and chemistry (**c**, **d**) benchmarks. Dark segments: relayed runs in which the agent also raised a concern; light segments: relayed without a concern; black ticks: share of all runs raising a concern. Numbers give the relay rate (%).

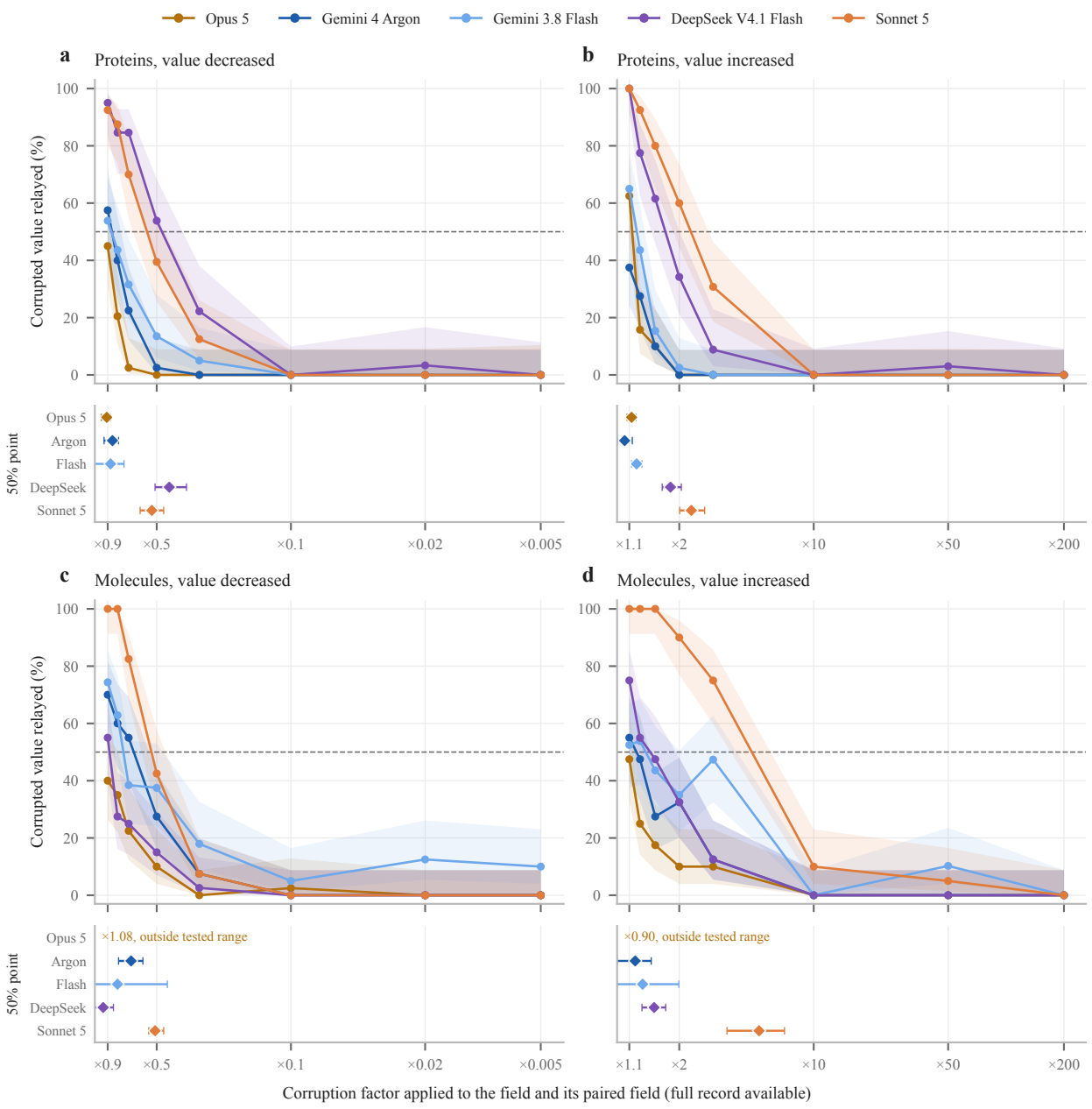


Figure 3: **Override boundaries.** Share of runs relaying a corrupted value as the corruption grows, with the full record available and the paired field scaled alike, for proteins (**a**, value decreased; **b**, increased) and molecules (**c**, decreased; **d**, increased). Bands: 95% Wilson intervals (30–40 tasks per point after excluding runs without a parseable answer). Strips below each panel: fitted factor at which half of runs relay the value, with its 95% interval; estimates outside the tested range are given as numbers.

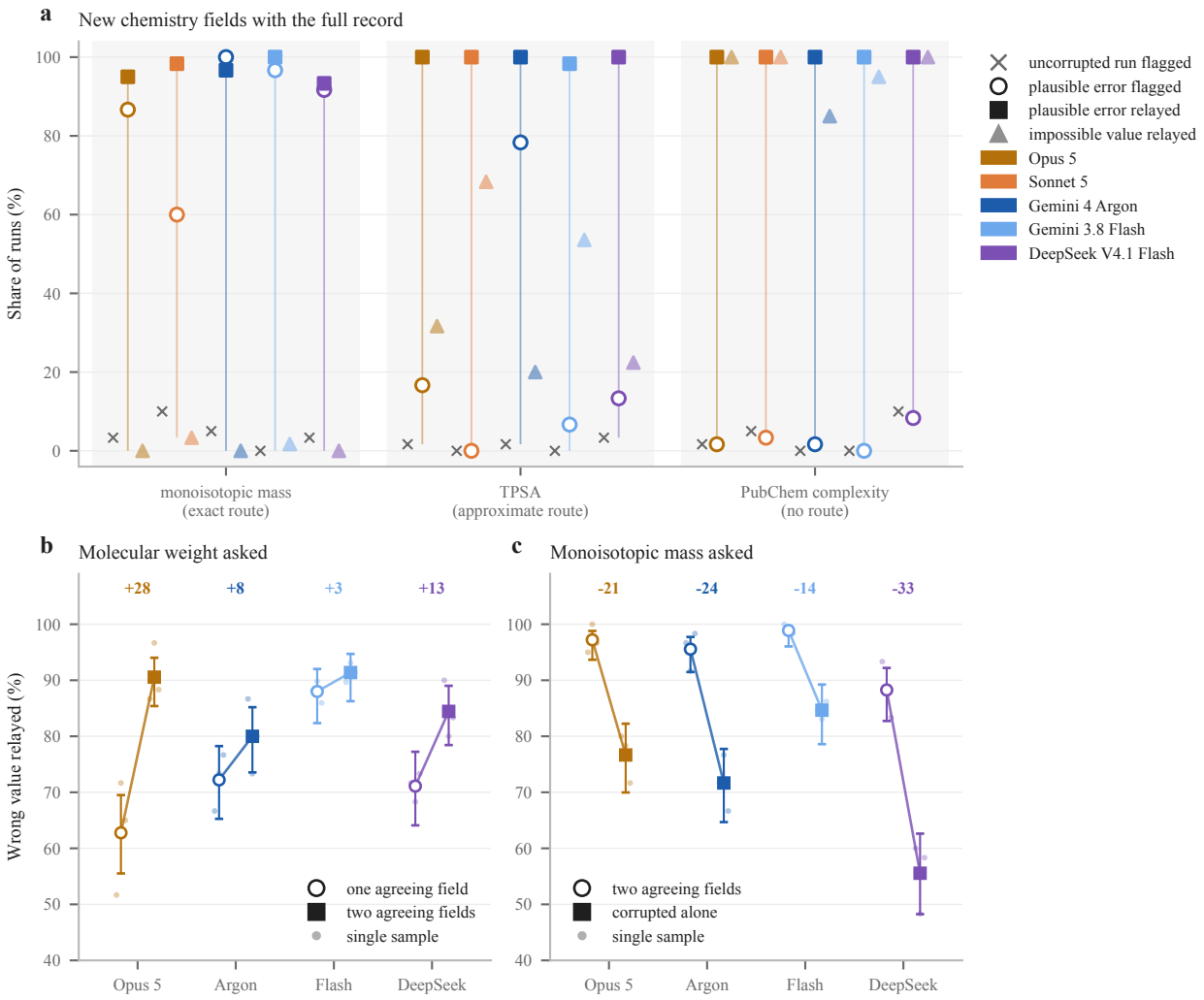


Figure 4: **A derivation route raises detection; agreement decides override. a**, New chemistry fields with the full record, per model: uncorrupted runs raising a concern (crosses); plausible-error runs raising a concern (open circles) and relaying the value (squares); impossible-value runs relaying it (triangles). **b**, Molecular weight asked: relay when the corrupted weight agreed with one or with two other fields. **c**, Monoisotopic mass asked: relay when the corrupted mass agreed with two other fields or was changed alone. Large markers pool three samples (95% Wilson intervals); small dots are single samples; numbers give the change in percentage points.

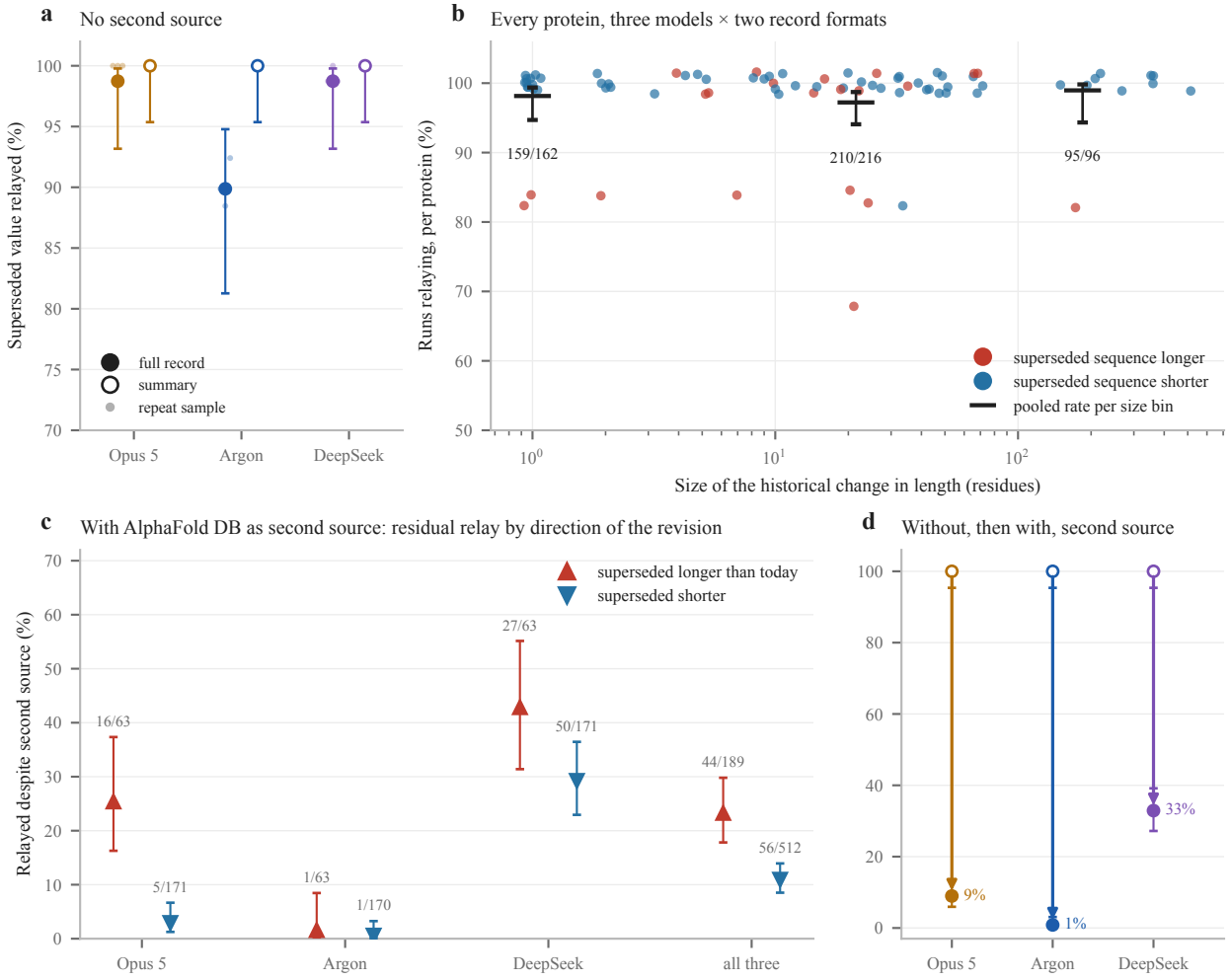


Figure 5: **Real, historically corrected UniProt errors. a**, Relay of the superseded value with the full (filled) and summary (open) record; small dots, repeat samples with the full record. **b**, For each of the 79 proteins, the share of six model–format runs relaying the superseded value against the size of the historical change in length, coloured by whether the superseded sequence was longer or shorter than today's; black bars, pooled rate per size bin with 95% interval. **c**, With AlphaFold DB as second source, residual relay on covered proteins by direction of the revision (three samples pooled). **d**, Relay without (open) and with (filled) the second source.

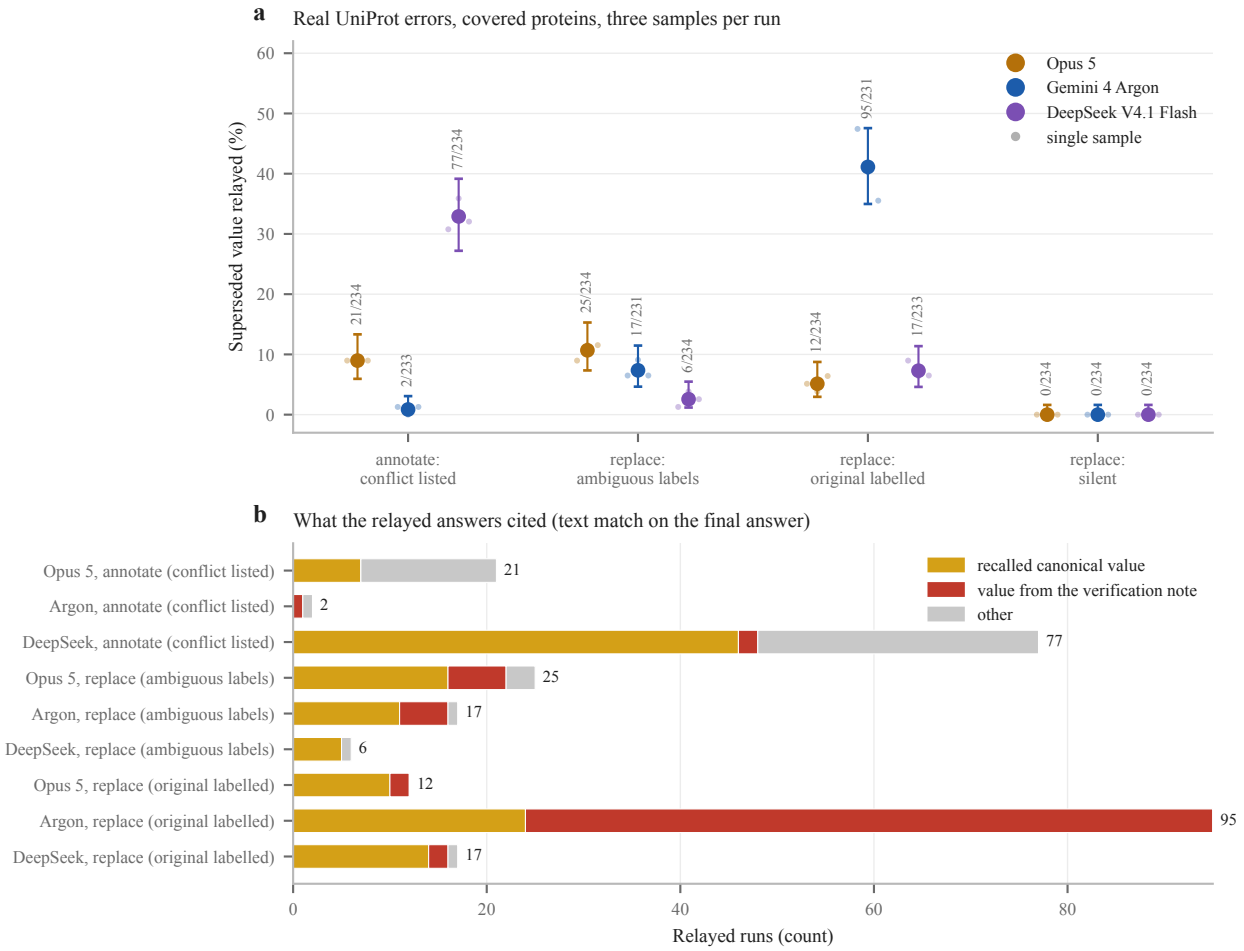


Figure 6: **Correcting the record works; showing the conflict does not. a**, Relay of superseded values on covered proteins under four presentations of the second source (three samples per run; 95% Wilson intervals; small dots, single samples; counts above). **b**, What the relayed answers cited, by a text match on the final answer: a recalled canonical value, the original or observed value shown in the verification note, or neither.